# JavaScript Callbacks

Back to JS page

Table of Contents



## What is a Callback Function?

A **callback** is a function passed as an argument to another function.

This technique allows a function to call another function after a task has been completed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is a Callback Function?</h4>
<p id="demo"></p>

<script>
function myCalculator(num, myCallback) {
  let result = num * 2;
  myCallback(result);
}

function displayResult(result) {
  document.getElementById("demo").innerHTML = "Result: " + result;
}

myCalculator(5, displayResult);
</script>

</body>
</html>
```

### Example 1

**Result** View Example

## Event Handling

Callbacks are commonly used in event handlers. The `addEventListener()` method takes a callback function as its second parameter:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Event Handling</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
function handleClick() {
  document.getElementById("demo").innerHTML = "Button was clicked!";
}

document.getElementById("myBtn").addEventListener("click", handleClick);
</script>

</body>
</html>
```

### Example 2

**Result** View Example

## Asynchronous Operations

Callbacks are essential for asynchronous operations. The `setTimeout()` function runs a callback after a delay:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Asynchronous Operations</h4>
<p id="demo"></p>

<script>
function showMessage() {
  document.getElementById("demo").innerHTML = "This message appears after 2 seconds!";
}

setTimeout(showMessage, 2000);
document.getElementById("demo").innerHTML = "Waiting...";
</script>

</body>
</html>
```

### Example 3

**Result** View Example

## Array Methods

Array methods like `forEach()` use callbacks to execute a function on each element:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Array Methods (forEach)</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];
let text = "";

numbers.forEach(function(value, index) {
  text += "Index " + index + ": " + value + "<br>";
});

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

### Example 4

**Result** View Example

## Sequence Control

Callbacks can be used to control the sequence of function execution, ensuring one task completes before the next begins:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sequence Control</h4>
<p id="demo"></p>

<script>
function step1(callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 1 complete<br>";
    callback();
  }, 1000);
}

function step2(callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 2 complete<br>";
    callback();
  }, 1000);
}

function step3() {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 3 complete<br>";
    document.getElementById("demo").innerHTML += "All steps finished!";
  }, 1000);
}

// Chain the callbacks
step1(function() {
  step2(function() {
    step3();
  });
});
</script>

</body>
</html>
```

## Example 5

**Result** View Example

## Document

Document in project

You can Download PDF file.

## Reference

- W3Schools JavaScript Callbacks